NLP Unit 5
Practice Programs

Chatbot • Question Answering • Summarization • Machine Translation

Generated on: May 03, 2026 at 11:27 PM

Implementation and Results

Table of Contents

Program 1. CHATBOT

Source Code:

```
"""
================================================================================
  NLP Unit 5 - Practice Program 1: CHATBOT
================================================================================
  A rule-based + TF-IDF retrieval chatbot that can answer questions about
  common topics using cosine similarity for response selection.
================================================================================
"""

import re
import random
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# 1. Knowledge Base (corpus of documents the chatbot can reference)
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
KNOWLEDGE_BASE = [
    "Natural Language Processing (NLP) is a subfield of artificial intelligence that focuses on the interaction between computers and h
    "Machine learning is a subset of artificial intelligence that provides systems the ability to automatically learn and improve from
    "Deep learning is a subset of machine learning that uses neural networks with many layers to learn representations of data.",
    "A chatbot is a software application used to conduct an online chat conversation via text or text-to-speech.",
    "Python is a high-level, interpreted programming language known for its simplicity and readability.",
    "TensorFlow and PyTorch are popular deep learning frameworks used for building neural networks.",
    "Transformers are a type of neural network architecture that has revolutionized NLP tasks like translation and summarization.",
    "BERT (Bidirectional Encoder Representations from Transformers) is a pre-trained language model by Google.",
    "GPT (Generative Pre-trained Transformer) is a family of large language models developed by OpenAI.",
    "Tokenization is the process of breaking text into smaller units called tokens, such as words or subwords.",
    "Word embeddings are dense vector representations of words that capture semantic meaning.",
    "Sentiment analysis is the process of determining the emotional tone behind a series of words.",
    "Named Entity Recognition (NER) identifies and classifies named entities in text into predefined categories.",
    "Text classification is the task of assigning predefined categories to text documents.",
    "Recurrent Neural Networks (RNNs) are a class of neural networks designed for sequential data processing.",
]

# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# 2. Rule-Based Response Patterns (for greetings &amp;amp; small talk)
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
RULE_PATTERNS = {
    r"\b(hi|hello|hey|greetings)\b": [
        "Hello! How can I help you today?",
        "Hi there! What would you like to know?",
        "Hey! I'm your NLP chatbot. Ask me anything!",
    ],
    r"\b(bye|goodbye|exit|quit)\b": [
        "Goodbye! Have a great day!",
        "See you later! Feel free to come back anytime.",
        "Bye! It was nice chatting with you.",
    ],
    r"\b(thanks|thank you|thankyou)\b": [
        "You're welcome!",
        "Happy to help!",
        "Glad I could assist!",
    ],
    r"\b(how are you|how do you do)\b": [
        "I'm just a chatbot, but I'm doing great! How can I help?",
        "I'm functioning perfectly! What can I do for you?",
    ],
    r"\b(your name|who are you)\b": [
        "I'm an NLP Practice Chatbot built for Unit 5!",
        "I'm a retrieval-based chatbot. Ask me about NLP topics!",
    ],
}


# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# 3. Text Preprocessing
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def preprocess(text):
    """Lowercase and remove special characters."""
    text = text.lower().strip()
    text = re.sub(r"[^a-z0-9\s]", "", text)
    return text


# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# 4. Rule-Based Matching
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def rule_based_response(user_input):
```

```
    """Check if the user input matches any rule-based pattern."""
    processed = preprocess(user_input)
    for pattern, responses in RULE_PATTERNS.items():
        if re.search(pattern, processed):
            return random.choice(responses)
    return None


# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# 5. TF-IDF Retrieval-Based Response
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def retrieval_based_response(user_input, threshold=0.15):
    """
    Use TF-IDF vectorization and cosine similarity to find the most
    relevant response from the knowledge base.
    """
    # Combine knowledge base with user query
    corpus = KNOWLEDGE_BASE + [user_input]

    # Vectorize
    vectorizer = TfidfVectorizer(stop_words="english")
    tfidf_matrix = vectorizer.fit_transform(corpus)

    # Compute cosine similarity between user query and all KB entries
    user_vector = tfidf_matrix[-1]
    similarities = cosine_similarity(user_vector, tfidf_matrix[:-1]).flatten()

    # Get best match
    best_idx = np.argmax(similarities)
    best_score = similarities[best_idx]

    if best_score &gt;= threshold:
        return KNOWLEDGE_BASE[best_idx], best_score
    else:
        return None, best_score


# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# 6. Main Chatbot Logic
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def chatbot_response(user_input):
    """Generate a response using rule-based or retrieval-based approach."""
    # Try rule-based first
    response = rule_based_response(user_input)
    if response:
        return response, "Rule-Based", 1.0

    # Fall back to retrieval-based
    response, score = retrieval_based_response(user_input)
    if response:
        return response, "TF-IDF Retrieval", score

    # Default fallback
    return (
        "I'm sorry, I don't have information about that. "
        "Try asking about NLP, machine learning, or deep learning!",
        "Fallback",
        0.0,
    )


# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# 7. Demonstration / Results
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def run_demo():
    print("=" * 70)
    print("  NLP UNIT 5 — PRACTICE PROGRAM 1: CHATBOT")
    print("  Rule-Based + TF-IDF Retrieval Chatbot")
    print("=" * 70)

    test_inputs = [
        "Hello!",
        "What is NLP?",
        "Tell me about deep learning",
        "What are transformers?",
        "Explain tokenization",
        "What is BERT?",
        "How are you?",
        "What is sentiment analysis?",
        "Tell me about word embeddings",
        "What is the weather today?",
        "Thank you!",
        "Goodbye",
    ]

    print("\n--- Chatbot Conversation Demo ---\n")
    for user_input in test_inputs:
        response, method, score = chatbot_response(user_input)
        print(f"  User   : {user_input}")
        print(f"  Bot    : {response}")
```

```
        print(f"  Method : {method} | Confidence: {score:.4f}")
        print("-" * 60)

    # Show TF-IDF similarity matrix for a sample query
    print("\n--- TF-IDF Similarity Analysis ---\n")
    sample_query = "What is natural language processing?"
    corpus = KNOWLEDGE_BASE + [sample_query]
    vectorizer = TfidfVectorizer(stop_words="english")
    tfidf_matrix = vectorizer.fit_transform(corpus)
    similarities = cosine_similarity(tfidf_matrix[-1], tfidf_matrix[:-1]).flatten()

    print(f"  Query: '{sample_query}'\n")
    print(f"  {'Rank':&lt;6} {'Similarity':&lt;12} {'Knowledge Base Entry'}")
    print(f"  {'■'*6} {'■'*12} {'■'*50}")
    ranked = sorted(enumerate(similarities), key=lambda x: x[1], reverse=True)
    for rank, (idx, sim) in enumerate(ranked[:5], 1):
        entry = KNOWLEDGE_BASE[idx][:60] + "..." if len(KNOWLEDGE_BASE[idx]) &gt; 60 else KNOWLEDGE_BASE[idx]
        print(f"  {rank:&lt;6} {sim:&lt;12.4f} {entry}")

    print("\n" + "=" * 70)
    print("  Program completed successfully.")
    print("=" * 70)


if __name__ == "__main__":
    run_demo()
```

Program Output / Results:

```
======================================================================
  NLP UNIT 5 — PRACTICE PROGRAM 1: CHATBOT
  Rule-Based + TF-IDF Retrieval Chatbot
======================================================================

--- Chatbot Conversation Demo ---

  User   : Hello!
  Bot    : Hey! I'm your NLP chatbot. Ask me anything!
  Method : Rule-Based | Confidence: 1.0000
------------------------------------------------------------
  User   : What is NLP?
  Bot    : Transformers are a type of neural network architecture that has revolutionized NLP tasks like translation and summ
  Method : TF-IDF Retrieval | Confidence: 0.2481
------------------------------------------------------------
  User   : Tell me about deep learning
  Bot    : Deep learning is a subset of machine learning that uses neural networks with many layers to learn representations
  Method : TF-IDF Retrieval | Confidence: 0.3624
------------------------------------------------------------
  User   : What are transformers?
  Bot    : BERT (Bidirectional Encoder Representations from Transformers) is a pre-trained language model by Google.
  Method : TF-IDF Retrieval | Confidence: 0.2716
------------------------------------------------------------
  User   : Explain tokenization
  Bot    : Tokenization is the process of breaking text into smaller units called tokens, such as words or subwords.
  Method : TF-IDF Retrieval | Confidence: 0.1947
------------------------------------------------------------
  User   : What is BERT?
  Bot    : BERT (Bidirectional Encoder Representations from Transformers) is a pre-trained language model by Google.
  Method : TF-IDF Retrieval | Confidence: 0.3052
------------------------------------------------------------
  User   : How are you?
  Bot    : I'm just a chatbot, but I'm doing great! How can I help?
  Method : Rule-Based | Confidence: 1.0000
------------------------------------------------------------
  User   : What is sentiment analysis?
  Bot    : Sentiment analysis is the process of determining the emotional tone behind a series of words.
  Method : TF-IDF Retrieval | Confidence: 0.4695
------------------------------------------------------------
  User   : Tell me about word embeddings
  Bot    : Word embeddings are dense vector representations of words that capture semantic meaning.
  Method : TF-IDF Retrieval | Confidence: 0.3439
------------------------------------------------------------
  User   : What is the weather today?
  Bot    : I'm sorry, I don't have information about that. Try asking about NLP, machine learning, or deep learning!
  Method : Fallback | Confidence: 0.0000
------------------------------------------------------------
  User   : Thank you!
  Bot    : Glad I could assist!
  Method : Rule-Based | Confidence: 1.0000
------------------------------------------------------------
```

```
  User   : Goodbye
  Bot    : Goodbye! Have a great day!
  Method : Rule-Based | Confidence: 1.0000
------------------------------------------------------------

--- TF-IDF Similarity Analysis ---

  Query: 'What is natural language processing?'

  Rank   Similarity   Knowledge Base Entry
  ■■■■■■ ■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  1      0.6254       Natural Language Processing (NLP) is a subfield of artificia...
  2      0.1382       Recurrent Neural Networks (RNNs) are a class of neural netwo...
  3      0.1097       BERT (Bidirectional Encoder Representations from Transformer...
  4      0.1089       Python is a high-level, interpreted programming language kno...
  5      0.1003       GPT (Generative Pre-trained Transformer) is a family of larg...

======================================================================
  Program completed successfully.
======================================================================
```

Program 2. QUESTION ANSWERING

Source Code:

```
"""
================================================================================
  NLP Unit 5 - Practice Program 2: QUESTION ANSWERING
================================================================================
  An extractive QA system using TF-IDF for retrieval and sentence extraction.
================================================================================
"""

import re
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

CONTEXTS = {
    "nlp": "Natural Language Processing (NLP) is a branch of artificial intelligence that deals with the interaction between computers
    "machine_learning": "Machine learning is a method of data analysis that automates analytical model building. It is a branch of arti
    "deep_learning": "Deep learning is a subset of machine learning that uses artificial neural networks with multiple layers to model
}


def preprocess_text(text):
    return re.sub(r"\s+", " ", text.strip())


def split_into_sentences(text):
    sentences = re.split(r"(?&lt;=[.!?])\s+", text)
    return [s.strip() for s in sentences if s.strip()]


def retrieve_relevant_context(question, contexts):
    context_texts = [preprocess_text(v) for v in contexts.values()]
    context_keys = list(contexts.keys())
    corpus = context_texts + [question]
    vectorizer = TfidfVectorizer(stop_words="english")
    tfidf_matrix = vectorizer.fit_transform(corpus)
    similarities = cosine_similarity(tfidf_matrix[-1], tfidf_matrix[:-1]).flatten()
    best_idx = np.argmax(similarities)
    return context_keys[best_idx], context_texts[best_idx], similarities[best_idx]


def extract_answer(question, context, top_k=2):
    sentences = split_into_sentences(context)
    if not sentences:
        return "No answer found.", []
    corpus = sentences + [question]
    vectorizer = TfidfVectorizer(stop_words="english")
    tfidf_matrix = vectorizer.fit_transform(corpus)
    similarities = cosine_similarity(tfidf_matrix[-1], tfidf_matrix[:-1]).flatten()
    top_indices = np.argsort(similarities)[::-1][:top_k]
    results = [(sentences[idx], similarities[idx]) for idx in top_indices]
    answer = " ".join([sentences[idx] for idx in sorted(top_indices)])
    return answer, results


def answer_question(question, contexts=CONTEXTS):
    topic, context, doc_score = retrieve_relevant_context(question, contexts)
    answer, sentence_scores = extract_answer(question, context)
    return {
        "question": question, "topic": topic,
        "doc_relevance": doc_score, "answer": answer,
        "sentence_scores": sentence_scores,
    }


def run_demo():
    print("=" * 70)
    print("  NLP UNIT 5 — PRACTICE PROGRAM 2: QUESTION ANSWERING")
    print("  TF-IDF Retrieval + Extractive QA System")
    print("=" * 70)

    test_questions = [
        "What is Natural Language Processing?",
        "What are the types of machine learning?",
        "What is deep learning used for?",
        "What are transformer models?",
        "How do neural networks learn features?",
        "What is supervised learning?",
        "What are CNNs used for?",
    ]
    print("\n--- TF-IDF Based Extractive QA ---\n")
```

```
    for question in test_questions:
        result = answer_question(question)
        print(f"  Q: {result['question']}")
        print(f"  Topic: {result['topic']} (relevance: {result['doc_relevance']:.4f})")
        answer_display = result['answer'][:150]
        print(f"  A: {answer_display}...")
        if result['sentence_scores']:
            print(f"  Top sentence score: {result['sentence_scores'][0][1]:.4f}")
        print("-" * 60)

    print("\n--- Detailed Analysis ---\n")
    question = "What are the applications of NLP?"
    result = answer_question(question)
    print(f"  Question: {question}")
    print(f"  Retrieved Topic: {result['topic']}")
    print(f"  Document Relevance Score: {result['doc_relevance']:.4f}\n")
    print("  Candidate Sentences (ranked by relevance):")
    for i, (sent, score) in enumerate(result['sentence_scores'], 1):
        display = sent[:80]
        print(f"    {i}. [{score:.4f}] {display}...")
    print(f"\n  Final Answer: {result['answer'][:200]}")

    # Transformer-based QA
    print("\n--- Transformer-Based QA (Hugging Face) ---\n")
    try:
        from transformers import pipeline
        qa_pipeline = pipeline("question-answering",
                               model="distilbert-base-cased-distilled-squad")
        context = preprocess_text(CONTEXTS["nlp"])
        for q in ["What is NLP?", "What are the applications of NLP?"]:
            r = qa_pipeline(question=q, context=context)
            print(f"  Q: {q}")
            print(f"  A: {r['answer']}  (score: {r['score']:.4f})")
            print("-" * 60)
    except Exception as e:
        print(f"  [INFO] Transformer QA skipped: {e}")
        print("  Install with: pip install transformers torch\n")

    print("\n" + "=" * 70)
    print("  Program completed successfully.")
    print("=" * 70)


if __name__ == "__main__":
    run_demo()
```

Program Output / Results:

```
======================================================================
  NLP UNIT 5 — PRACTICE PROGRAM 2: QUESTION ANSWERING
  TF-IDF Retrieval + Extractive QA System
======================================================================

--- TF-IDF Based Extractive QA ---

  Q: What is Natural Language Processing?
  Topic: nlp (relevance: 0.3426)
  A: Natural Language Processing (NLP) is a branch of artificial intelligence that deals with the interaction between compute
  Top sentence score: 0.5988
------------------------------------------------------------
  Q: What are the types of machine learning?
  Topic: machine_learning (relevance: 0.3590)
  A: Machine learning is a method of data analysis that automates analytical model building. There are three main types of ma
  Top sentence score: 0.6271
------------------------------------------------------------
  Q: What is deep learning used for?
  Topic: deep_learning (relevance: 0.2702)
  A: Deep learning is a subset of machine learning that uses artificial neural networks with multiple layers to model complex
  Top sentence score: 0.1670
------------------------------------------------------------
  Q: What are transformer models?
  Topic: nlp (relevance: 0.1721)
  A: NLP combines computational linguistics with statistical, machine learning, and deep learning models. NLP has evolved sig
  Top sentence score: 0.3941
------------------------------------------------------------
  Q: How do neural networks learn features?
  Topic: deep_learning (relevance: 0.2980)
  A: Deep learning is a subset of machine learning that uses artificial neural networks with multiple layers to model complex
  Top sentence score: 0.2795
------------------------------------------------------------
  Q: What is supervised learning?
  Topic: machine_learning (relevance: 0.4212)
```

```
  A: There are three main types of machine learning: supervised learning, unsupervised learning, and reinforcement learning.
  Top sentence score: 0.5757
------------------------------------------------------------
  Q: What are CNNs used for?
  Topic: deep_learning (relevance: 0.0548)
  A: Deep learning architectures include Convolutional Neural Networks (CNNs) for image processing, Recurrent Neural Networks
  Top sentence score: 0.1213
------------------------------------------------------------

--- Detailed Analysis ---

  Question: What are the applications of NLP?
  Retrieved Topic: nlp
  Document Relevance Score: 0.3297

  Candidate Sentences (ranked by relevance):
    1. [0.2872] Applications of NLP include machine translation, sentiment analysis, chatbots, t...
    2. [0.0753] NLP has evolved significantly with the introduction of transformer models like B...

  Final Answer: Applications of NLP include machine translation, sentiment analysis, chatbots, text summarization, and speech

--- Transformer-Based QA (Hugging Face) ---

  [INFO] Transformer QA skipped: No module named 'transformers'
  Install with: pip install transformers torch


======================================================================
  Program completed successfully.
======================================================================
```

Program 3. TEXT SUMMARIZATION

Source Code:

```
"""
================================================================================
  NLP Unit 5 - Practice Program 3: TEXT SUMMARIZATION
================================================================================
  Implements both Extractive and Abstractive summarization techniques.
  - Extractive: TF-IDF sentence scoring
  - Abstractive: Hugging Face Transformer pipeline (optional)
================================================================================
"""

import re
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# 1. Sample Documents for Summarization
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
DOCUMENTS = {
    "AI_Article": """
Artificial Intelligence (AI) has transformed numerous industries and continues
to shape the future of technology. Machine learning, a subset of AI, enables
computers to learn from data without explicit programming. Deep learning, which
uses neural networks with many layers, has achieved remarkable results in image
recognition, natural language processing, and game playing. The development of
transformer architectures has revolutionized NLP tasks. Models like BERT and GPT
have set new benchmarks in text understanding and generation. AI is being applied
in healthcare for disease diagnosis, in finance for fraud detection, in
transportation for autonomous vehicles, and in education for personalized learning.
However, AI also raises ethical concerns about privacy, bias, and job displacement.
Researchers are actively working on making AI systems more transparent, fair, and
accountable. The future of AI depends on responsible development and deployment
of these powerful technologies.
""",
    "Climate_Article": """
Climate change is one of the most pressing challenges facing humanity today.
Global temperatures have risen by approximately 1.1 degrees Celsius since
pre-industrial times. The burning of fossil fuels is the primary driver of
greenhouse gas emissions. These emissions trap heat in the atmosphere, leading
to rising temperatures, melting ice caps, and rising sea levels. Extreme weather
events such as hurricanes, droughts, and floods are becoming more frequent and
severe. The Paris Agreement aims to limit global warming to 1.5 degrees Celsius
above pre-industrial levels. Countries around the world are implementing
renewable energy solutions including solar, wind, and hydroelectric power.
Carbon capture technologies are being developed to remove CO2 from the
atmosphere. Individual actions such as reducing energy consumption, using public
transportation, and eating less meat can also contribute to fighting climate
change. Scientists emphasize that immediate and collective action is necessary
to prevent the worst impacts of climate change.
""",
}


# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# 2. Text Preprocessing
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def preprocess(text):
    text = re.sub(r"\s+", " ", text.strip())
    return text


def split_sentences(text):
    sentences = re.split(r"(?&lt;=[.!?])\s+", text)
    return [s.strip() for s in sentences if len(s.strip()) &gt; 10]


# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# 3. Extractive Summarization using TF-IDF
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def extractive_summarize_tfidf(text, num_sentences=3):
    """
    Extractive summarization by scoring sentences based on their
    TF-IDF similarity to the overall document.
    """
    clean_text = preprocess(text)
    sentences = split_sentences(clean_text)

    if len(sentences) &lt;= num_sentences:
        return " ".join(sentences), sentences, [1.0] * len(sentences)
    # Vectorize all sentences
```

```
    vectorizer = TfidfVectorizer(stop_words="english")
    tfidf_matrix = vectorizer.fit_transform(sentences)

    # Compute document vector (mean of all sentence vectors)
    doc_vector = np.asarray(tfidf_matrix.mean(axis=0))

    # Score each sentence by similarity to document
    scores = []
    for i in range(len(sentences)):
        sim = cosine_similarity(tfidf_matrix[i], doc_vector).flatten()[0]
        scores.append(sim)

    scores = np.array(scores)

    # Select top sentences (maintain original order)
    top_indices = np.argsort(scores)[::-1][:num_sentences]
    top_indices_sorted = sorted(top_indices)

    summary_sentences = [sentences[i] for i in top_indices_sorted]
    summary = " ".join(summary_sentences)

    return summary, sentences, scores


# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# 4. Extractive Summarization using TextRank (simplified)
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def extractive_summarize_textrank(text, num_sentences=3, damping=0.85, iterations=30):
    """
    TextRank-based extractive summarization.
    Builds a sentence similarity graph and ranks using PageRank-like algorithm.
    """
    clean_text = preprocess(text)
    sentences = split_sentences(clean_text)

    if len(sentences) &lt;= num_sentences:
        return " ".join(sentences)

    # Build similarity matrix
    vectorizer = TfidfVectorizer(stop_words="english")
    tfidf_matrix = vectorizer.fit_transform(sentences)
    sim_matrix = cosine_similarity(tfidf_matrix)

    # Normalize similarity matrix
    np.fill_diagonal(sim_matrix, 0)
    row_sums = sim_matrix.sum(axis=1, keepdims=True)
    row_sums[row_sums == 0] = 1
    norm_matrix = sim_matrix / row_sums

    # PageRank iteration
    n = len(sentences)
    scores = np.ones(n) / n
    for _ in range(iterations):
        scores = (1 - damping) / n + damping * norm_matrix.T.dot(scores)

    # Select top sentences
    top_indices = np.argsort(scores)[::-1][:num_sentences]
    top_indices_sorted = sorted(top_indices)
    summary = " ".join([sentences[i] for i in top_indices_sorted])
    return summary, scores


# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# 5. Evaluation Metrics
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def compute_compression_ratio(original, summary):
    orig_words = len(original.split())
    summ_words = len(summary.split())
    return summ_words / orig_words if orig_words &gt; 0 else 0


def simple_rouge_1(reference, hypothesis):
    """Compute simple ROUGE-1 (unigram overlap) F1 score."""
    ref_tokens = set(reference.lower().split())
    hyp_tokens = set(hypothesis.lower().split())
    overlap = ref_tokens &amp; hyp_tokens
    if not hyp_tokens or not ref_tokens:
        return 0.0
    precision = len(overlap) / len(hyp_tokens)
    recall = len(overlap) / len(ref_tokens)
    if precision + recall == 0:
        return 0.0
    f1 = 2 * precision * recall / (precision + recall)
    return f1


# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# 6. Demonstration / Results
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def run_demo():
```

```
    print("=" * 70)
    print("  NLP UNIT 5 – PRACTICE PROGRAM 3: TEXT SUMMARIZATION")
    print("  Extractive (TF-IDF + TextRank) Summarization")
    print("=" * 70)

    for doc_name, doc_text in DOCUMENTS.items():
        clean = preprocess(doc_text)
        print(f"\n{'■'*70}")
        print(f"  Document: {doc_name}")
        print(f"  Original Length: {len(clean.split())} words")
        print(f"{'■'*70}")
        print(f"\n  Original Text:\n  {clean[:300]}...\n")

        # Method 1: TF-IDF Extractive
        summary_tfidf, sentences, scores = extractive_summarize_tfidf(clean, num_sentences=3)
        ratio_tfidf = compute_compression_ratio(clean, summary_tfidf)

        print("  --- Method 1: TF-IDF Extractive Summarization ---")
        print(f"  Summary: {summary_tfidf[:250]}...")
        print(f"  Summary Length: {len(summary_tfidf.split())} words")
        print(f"  Compression Ratio: {ratio_tfidf:.2%}")
        print(f"\n  Sentence Scores:")
        for i, (sent, score) in enumerate(zip(sentences, scores)):
            marker = " ★" if score &gt;= sorted(scores)[-3] else ""
            print(f"    [{score:.4f}]{marker} {sent[:70]}...")
        print()

        # Method 2: TextRank
        summary_tr, tr_scores = extractive_summarize_textrank(clean, num_sentences=3)
        ratio_tr = compute_compression_ratio(clean, summary_tr)

        print("  --- Method 2: TextRank Summarization ---")
        print(f"  Summary: {summary_tr[:250]}...")
        print(f"  Summary Length: {len(summary_tr.split())} words")
        print(f"  Compression Ratio: {ratio_tr:.2%}")

        # ROUGE-1 comparison
        rouge_score = simple_rouge_1(summary_tfidf, summary_tr)
        print(f"\n  ROUGE-1 F1 (TF-IDF vs TextRank): {rouge_score:.4f}")

    # Abstractive summarization
    print(f"\n{'■'*70}")
    print("  --- Abstractive Summarization (Hugging Face) ---")
    print(f"{'■'*70}\n")
    try:
        from transformers import pipeline
        summarizer = pipeline("summarization", model="sshleifer/distilbart-cnn-12-6")
        text = preprocess(DOCUMENTS["AI_Article"])
        result = summarizer(text, max_length=80, min_length=25, do_sample=False)
        print(f"  Abstractive Summary: {result[0]['summary_text']}")
    except Exception as e:
        print(f"  [INFO] Abstractive summarization skipped: {e}")
        print("  Install with: pip install transformers torch")

    print("\n" + "=" * 70)
    print("  Program completed successfully.")
    print("=" * 70)


if __name__ == "__main__":
    run_demo()
```

Program Output / Results:

```
======================================================================
  NLP UNIT 5 – PRACTICE PROGRAM 3: TEXT SUMMARIZATION
  Extractive (TF-IDF + TextRank) Summarization
======================================================================

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  Document: AI_Article
  Original Length: 139 words
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

  Original Text:
  Artificial Intelligence (AI) has transformed numerous industries and continues to shape the future of technology. Machine l

  --- Method 1: TF-IDF Extractive Summarization ---
  Summary: Artificial Intelligence (AI) has transformed numerous industries and continues to shape the future of technology.
  Summary Length: 44 words
  Compression Ratio: 31.65%

  Sentence Scores:
    [0.3825] ★ Artificial Intelligence (AI) has transformed numerous industries and c...
```

```
    [0.3843] ★ Machine learning, a subset of AI, enables computers to learn from data...
    [0.3325] Deep learning, which uses neural networks with many layers, has achiev...
    [0.3420] The development of transformer architectures has revolutionized NLP ta...
    [0.3067] Models like BERT and GPT have set new benchmarks in text understanding...
    [0.3751] AI is being applied in healthcare for disease diagnosis, in finance fo...
    [0.3584] However, AI also raises ethical concerns about privacy, bias, and job ...
    [0.3559] Researchers are actively working on making AI systems more transparent...
    [0.4237] ★ The future of AI depends on responsible development and deployment of ...

  --- Method 2: TextRank Summarization ---
  Summary: Artificial Intelligence (AI) has transformed numerous industries and continues to shape the future of technology.
  Summary Length: 44 words
  Compression Ratio: 31.65%

  ROUGE-1 F1 (TF-IDF vs TextRank): 1.0000

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  Document: Climate_Article
  Original Length: 150 words
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

  Original Text:
  Climate change is one of the most pressing challenges facing humanity today. Global temperatures have risen by approximatel

  --- Method 1: TF-IDF Extractive Summarization ---
  Summary: Global temperatures have risen by approximately 1.1 degrees Celsius since pre-industrial times. The Paris Agreemen
  Summary Length: 48 words
  Compression Ratio: 32.00%

  Sentence Scores:
    [0.3612] Climate change is one of the most pressing challenges facing humanity ...
    [0.4304] ★ Global temperatures have risen by approximately 1.1 degrees Celsius si...
    [0.3053] The burning of fossil fuels is the primary driver of greenhouse gas em...
    [0.3651] These emissions trap heat in the atmosphere, leading to rising tempera...
    [0.2853] Extreme weather events such as hurricanes, droughts, and floods are be...
    [0.4280] ★ The Paris Agreement aims to limit global warming to 1.5 degrees Celsiu...
    [0.3029] Countries around the world are implementing renewable energy solutions...
    [0.3067] Carbon capture technologies are being developed to remove CO2 from the...
    [0.3663] ★ Individual actions such as reducing energy consumption, using public t...
    [0.3533] Scientists emphasize that immediate and collective action is necessary...

  --- Method 2: TextRank Summarization ---
  Summary: Global temperatures have risen by approximately 1.1 degrees Celsius since pre-industrial times. The Paris Agreemen
  Summary Length: 48 words
  Compression Ratio: 32.00%

  ROUGE-1 F1 (TF-IDF vs TextRank): 1.0000

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  --- Abstractive Summarization (Hugging Face) ---
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

  [INFO] Abstractive summarization skipped: No module named 'transformers'
  Install with: pip install transformers torch

======================================================================
  Program completed successfully.
======================================================================
```

Program 4. MACHINE TRANSLATION

Source Code:

```
"""
================================================================================
  NLP Unit 5 - Practice Program 4: MACHINE TRANSLATION
================================================================================
  Implements a simple word-level statistical translation system and
  demonstrates Seq2Seq concepts for English-to-French translation.
================================================================================
"""

import re
import numpy as np
from collections import defaultdict

# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# 1. Parallel Corpus (English-French sentence pairs)
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
PARALLEL_CORPUS = [
    ("hello", "bonjour"),
    ("good morning", "bonjour"),
    ("how are you", "comment allez vous"),
    ("thank you", "merci"),
    ("goodbye", "au revoir"),
    ("yes", "oui"),
    ("no", "non"),
    ("please", "s il vous plait"),
    ("i am a student", "je suis un etudiant"),
    ("i love programming", "j aime la programmation"),
    ("the cat is on the table", "le chat est sur la table"),
    ("he is a good teacher", "il est un bon professeur"),
    ("she reads a book", "elle lit un livre"),
    ("we are learning french", "nous apprenons le francais"),
    ("the weather is nice today", "le temps est beau aujourd hui"),
    ("i like to eat pizza", "j aime manger de la pizza"),
    ("the dog is running", "le chien court"),
    ("they are playing football", "ils jouent au football"),
    ("this is a beautiful house", "c est une belle maison"),
    ("i want to learn machine learning", "je veux apprendre l apprentissage automatique"),
    ("natural language processing is interesting", "le traitement du langage naturel est interessant"),
    ("the book is on the table", "le livre est sur la table"),
    ("i am happy", "je suis heureux"),
    ("she is a doctor", "elle est medecin"),
    ("we love music", "nous aimons la musique"),
]


# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# 2. Text Preprocessing
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def preprocess(text):
    return re.sub(r"[^a-z\s']", "", text.lower().strip())


def tokenize(text):
    return preprocess(text).split()


# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# 3. Word-Level Translation Model (IBM Model 1 — Simplified)
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
class SimpleTranslationModel:
    """
    A simplified IBM Model 1 implementation for word alignment
    and translation probabilities using EM algorithm.
    """

    def __init__(self):
        self.trans_prob = defaultdict(lambda: defaultdict(float))
        self.word_dict = {}

    def train(self, parallel_corpus, iterations=10):
        """Train translation probabilities using EM algorithm."""
        # Initialize uniform probabilities
        target_vocab = set()
        for src, tgt in parallel_corpus:
            for w in tokenize(tgt):
                target_vocab.add(w)

        n_target = len(target_vocab)
        for src, tgt in parallel_corpus:
            src_tokens = tokenize(src)
```

```
            tgt_tokens = tokenize(tgt)
            for s in src_tokens:
                for t in tgt_tokens:
                    self.trans_prob[s][t] = 1.0 / n_target

        # EM iterations
        for iteration in range(iterations):
            counts = defaultdict(lambda: defaultdict(float))
            totals = defaultdict(float)

            for src, tgt in parallel_corpus:
                src_tokens = tokenize(src)
                tgt_tokens = tokenize(tgt)

                for s in src_tokens:
                    z = sum(self.trans_prob[s][t] for t in tgt_tokens)
                    for t in tgt_tokens:
                        c = self.trans_prob[s][t] / z if z &gt; 0 else 0
                        counts[s][t] += c
                        totals[s] += c

            # Update probabilities
            for s in counts:
                for t in counts[s]:
                    self.trans_prob[s][t] = counts[s][t] / totals[s] if totals[s] &gt; 0 else 0

        # Build word dictionary (best translation for each word)
        for s in self.trans_prob:
            best_t = max(self.trans_prob[s], key=self.trans_prob[s].get)
            self.word_dict[s] = (best_t, self.trans_prob[s][best_t])

    def translate_word(self, word):
        """Translate a single word."""
        word = word.lower()
        if word in self.word_dict:
            return self.word_dict[word]
        return (word, 0.0)  # unknown word

    def translate_sentence(self, sentence):
        """Translate a sentence word by word."""
        tokens = tokenize(sentence)
        translated = []
        details = []
        for token in tokens:
            trans, prob = self.translate_word(token)
            translated.append(trans)
            details.append((token, trans, prob))
        return " ".join(translated), details

    def get_top_translations(self, word, top_k=5):
        """Get top-k translation candidates for a word."""
        word = word.lower()
        if word not in self.trans_prob:
            return []
        probs = self.trans_prob[word]
        sorted_trans = sorted(probs.items(), key=lambda x: x[1], reverse=True)
        return sorted_trans[:top_k]


# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# 4. BLEU Score (simplified unigram)
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def simple_bleu(reference, hypothesis):
    """Compute simplified unigram BLEU score."""
    ref_tokens = tokenize(reference)
    hyp_tokens = tokenize(hypothesis)
    if not hyp_tokens:
        return 0.0
    ref_counts = defaultdict(int)
    for t in ref_tokens:
        ref_counts[t] += 1
    matches = 0
    hyp_counts = defaultdict(int)
    for t in hyp_tokens:
        hyp_counts[t] += 1
    for t in hyp_counts:
        matches += min(hyp_counts[t], ref_counts.get(t, 0))
    precision = matches / len(hyp_tokens)
    # Brevity penalty
    bp = min(1.0, len(hyp_tokens) / len(ref_tokens)) if ref_tokens else 0
    return bp * precision


# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# 5. Seq2Seq Concept Demonstration
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def demonstrate_seq2seq_concept():
    """Demonstrate Seq2Seq architecture concepts for MT."""
    print("\n  --- Seq2Seq Architecture Concepts ---\n")
    print("  Encoder-Decoder Architecture for Machine Translation:")
```

```
    print("  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■")
    print("  ■  INPUT: 'I love programming'                      ■")
    print("  ■                                                    ■")
    print("  ■  ENCODER (processes source language):              ■")
    print("  ■  [I] → [love] → [programming]                      ■")
    print("  ■   h1  →  h2   →   h3 (context vector)              ■")
    print("  ■                                                    ■")
    print("  ■  CONTEXT VECTOR: h3 (fixed-size representation)    ■")
    print("  ■                                                    ■")
    print("  ■  DECODER (generates target language):              ■")
    print("  ■  &lt;SOS&gt; → [j'] → [aime] → [la] → [programmation]    ■")
    print("  ■                                                    ■")
    print("  ■  OUTPUT: 'j aime la programmation'                 ■")
    print("  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■")
    print()
    print("  Key Components:")
    print("  • Encoder RNN/LSTM: Reads source sentence, produces context")
    print("  • Context Vector: Compressed representation of source")
    print("  • Decoder RNN/LSTM: Generates target sentence word by word")
    print("  • Attention Mechanism: Allows decoder to focus on relevant")
    print("    parts of the source sentence at each decoding step")
    print("  • Transformer: Replaces RNNs with self-attention for")
    print("    parallel processing and better long-range dependencies")


# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# 6. Demonstration / Results
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def run_demo():
    print("=" * 70)
    print("  NLP UNIT 5 — PRACTICE PROGRAM 4: MACHINE TRANSLATION")
    print("  Statistical Word-Level Translation (IBM Model 1)")
    print("=" * 70)

    # Train the model
    print("\n--- Training Translation Model ---\n")
    model = SimpleTranslationModel()
    model.train(PARALLEL_CORPUS, iterations=15)
    print(f"  Training corpus: {len(PARALLEL_CORPUS)} sentence pairs")
    print(f"  Vocabulary size: {len(model.word_dict)} source words")
    print("  Training completed (15 EM iterations).")

    # Show learned word translations
    print("\n--- Learned Word Translation Table ---\n")
    print(f"  {'English':&lt;20} {'French':&lt;20} {'Probability':&lt;12}")
    print(f"  {'■'*20} {'■'*20} {'■'*12}")
    sample_words = ["i", "the", "is", "love", "learning", "good",
                    "am", "book", "cat", "dog", "happy", "we"]
    for word in sample_words:
        if word in model.word_dict:
            trans, prob = model.word_dict[word]
            print(f"  {word:&lt;20} {trans:&lt;20} {prob:&lt;12.4f}")

    # Show top translation candidates
    print("\n--- Top Translation Candidates ---\n")
    for word in ["the", "is", "i"]:
        candidates = model.get_top_translations(word, top_k=5)
        cand_str = ", ".join([f"{t}({p:.3f})" for t, p in candidates])
        print(f"  '{word}' → {cand_str}")

    # Translate test sentences
    print("\n--- Translation Results ---\n")
    test_sentences = [
        ("i am happy", "je suis heureux"),
        ("the cat is on the table", "le chat est sur la table"),
        ("i love programming", "j aime la programmation"),
        ("she is a doctor", "elle est medecin"),
        ("we love music", "nous aimons la musique"),
        ("good morning", "bonjour"),
    ]

    print(f"  {'Source (English)':&lt;30} {'Translation':&lt;30} {'Reference':&lt;30} {'BLEU'}")
    print(f"  {'■'*30} {'■'*30} {'■'*30} {'■'*6}")
    for src, ref in test_sentences:
        translation, details = model.translate_sentence(src)
        bleu = simple_bleu(ref, translation)
        print(f"  {src:&lt;30} {translation:&lt;30} {ref:&lt;30} {bleu:.3f}")

    # Detailed word-by-word translation
    print("\n--- Detailed Word-by-Word Translation ---\n")
    test_sent = "i love programming"
    translation, details = model.translate_sentence(test_sent)
    print(f"  Source: '{test_sent}'")
    print(f"  Translation: '{translation}'\n")
    print(f"  {'Source Word':&lt;15} {'Target Word':&lt;15} {'Confidence':&lt;12}")
    print(f"  {'■'*15} {'■'*15} {'■'*12}")
    for src_w, tgt_w, prob in details:
        print(f"  {src_w:&lt;15} {tgt_w:&lt;15} {prob:&lt;12.4f}")

    # BLEU score analysis
```

```
    print("\n--- BLEU Score Analysis ---\n")
    total_bleu = 0
    count = 0
    for src, ref in test_sentences:
        trans, _ = model.translate_sentence(src)
        bleu = simple_bleu(ref, trans)
        total_bleu += bleu
        count += 1
    avg_bleu = total_bleu / count if count &gt; 0 else 0
    print(f"  Average BLEU Score: {avg_bleu:.4f}")
    print(f"  Total test sentences: {count}")

    # Seq2Seq concepts
    demonstrate_seq2seq_concept()

    # Transformer-based translation
    print("\n--- Transformer-Based Translation (Hugging Face) ---\n")
    try:
        from transformers import pipeline
        translator = pipeline("translation_en_to_fr",
                              model="Helsinki-NLP/opus-mt-en-fr")
        test = ["I love programming.", "The weather is nice today.",
                "Machine learning is interesting."]
        for t in test:
            result = translator(t)
            print(f"  EN: {t}")
            print(f"  FR: {result[0]['translation_text']}")
            print("-" * 50)
    except Exception as e:
        print(f"  [INFO] Transformer translation skipped: {e}")
        print("  Install: pip install transformers torch sentencepiece")

    print("\n" + "=" * 70)
    print("  Program completed successfully.")
    print("=" * 70)


if __name__ == "__main__":
    run_demo()
```

Program Output / Results:

```
======================================================================
  NLP UNIT 5 – PRACTICE PROGRAM 4: MACHINE TRANSLATION
  Statistical Word-Level Translation (IBM Model 1)
======================================================================

--- Training Translation Model ---

  Training corpus: 25 sentence pairs
  Vocabulary size: 55 source words
  Training completed (15 EM iterations).

--- Learned Word Translation Table ---

  English              French               Probability
  ■■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■
  i                    je                   0.5988
  the                  le                   0.9711
  is                   est                  0.7500
  love                 la                   0.9998
  learning             nous                 0.1250
  good                 bonjour              0.5000
  am                   je                   0.5000
  book                 livre                0.9998
  cat                  le                   0.1667
  dog                  le                   0.3333
  happy                je                   0.3333
  we                   nous                 0.9998

--- Top Translation Candidates ---

  'the' → le(0.971), est(0.027), sur(0.001), la(0.001), table(0.001)
  'is' → est(0.750), le(0.250), chien(0.000), court(0.000), sur(0.000)
  'i' → je(0.599), j(0.133), aime(0.133), la(0.133), suis(0.001)

--- Translation Results ---

  Source (English)              Translation                   Reference                     BLEU
  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■■■■■■
  i am happy                    je je je                      je suis heureux               0.333
```

```
  the cat is on the table        le le est le le le            le chat est sur la table       0.333
  i love programming             je la j                       j aime la programmation        0.500
  she is a doctor                elle est est elle             elle est medecin               0.500
  we love music                  nous la nous                  nous aimons la musique         0.500
  good morning                   bonjour bonjour               bonjour                        0.500

--- Detailed Word-by-Word Translation ---

  Source: 'i love programming'
  Translation: 'je la j'

  Source Word     Target Word     Confidence
  ■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■ ■■■■■■■■■■■■
  i               je              0.5988
  love            la              0.9998
  programming     j               0.2500

--- BLEU Score Analysis ---

  Average BLEU Score: 0.4444
  Total test sentences: 6

  --- Seq2Seq Architecture Concepts ---

  Encoder-Decoder Architecture for Machine Translation:
  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  ■  INPUT: 'I love programming'                      ■
  ■                                                   ■
  ■  ENCODER (processes source language):             ■
  ■  [I] → [love] → [programming]                     ■
  ■   h1  →  h2   →   h3 (context vector)             ■
  ■                                                   ■
  ■  CONTEXT VECTOR: h3 (fixed-size representation)   ■
  ■                                                   ■
  ■  DECODER (generates target language):             ■
  ■  &lt;SOS&gt; → [j'] → [aime] → [la] → [programmation]    ■
  ■                                                   ■
  ■  OUTPUT: 'j aime la programmation'                ■
  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

  Key Components:
  • Encoder RNN/LSTM: Reads source sentence, produces context
  • Context Vector: Compressed representation of source
  • Decoder RNN/LSTM: Generates target sentence word by word
  • Attention Mechanism: Allows decoder to focus on relevant
    parts of the source sentence at each decoding step
  • Transformer: Replaces RNNs with self-attention for
    parallel processing and better long-range dependencies

--- Transformer-Based Translation (Hugging Face) ---

  [INFO] Transformer translation skipped: No module named 'transformers'
  Install: pip install transformers torch sentencepiece

======================================================================
  Program completed successfully.
======================================================================
```